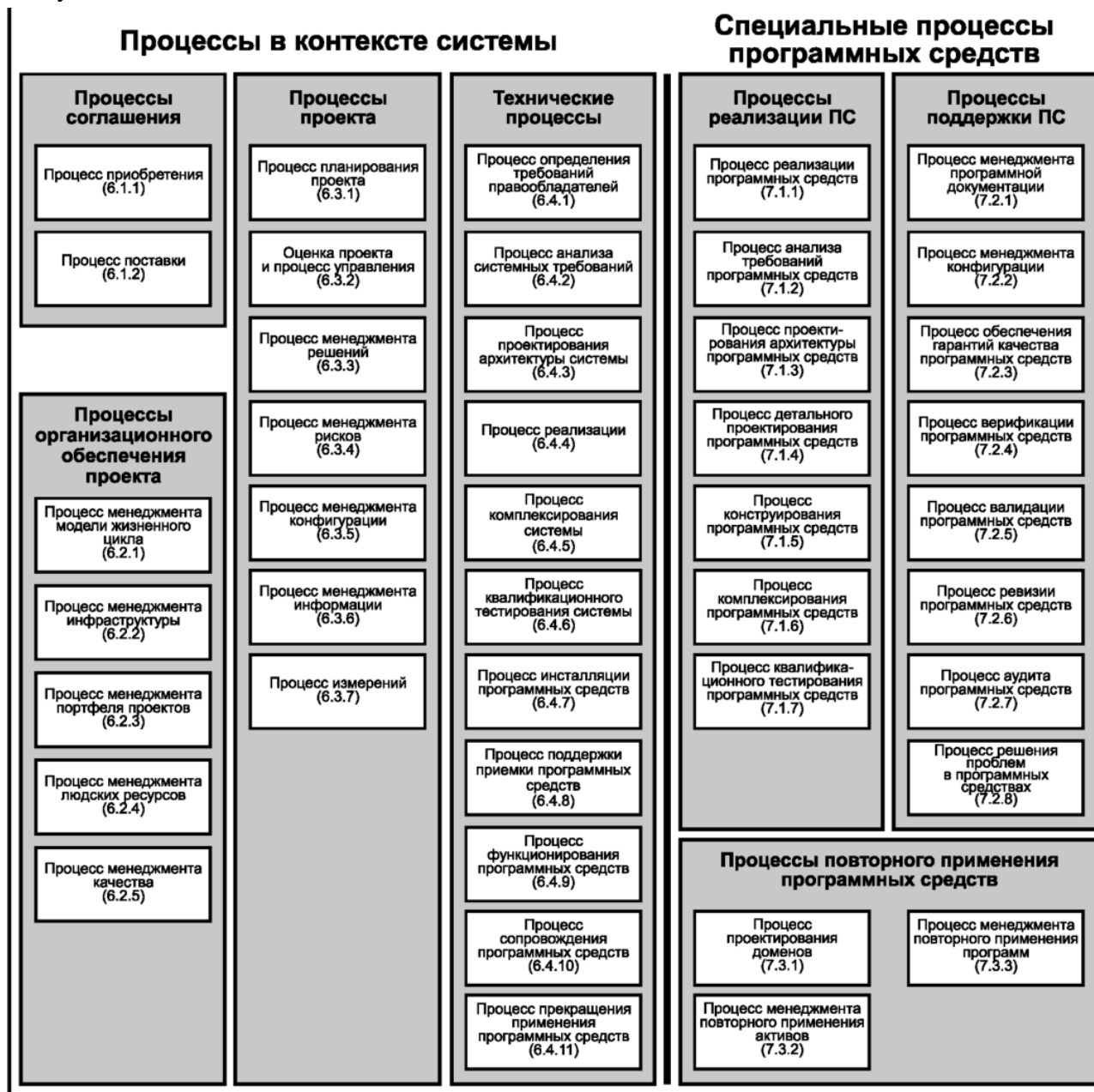


ОПИ ЭКЗ

1. ISO/IEC 12207:2010: Жизненный цикл ПО. Группы процессов ЖЦ.

Жизненный цикл ПО - время существования программы от задумки до вывода из эксплуатации



2. Модели ЖЦ (последовательная, инкрементная, эволюционная).

Последовательная - определены все требования, один этап разработки и тестов, результаты разработки не меняются

Инкрементная - продукт разбивается на неск частей по требованиям которые проходят этапы разработки тестов и тд по отдельности

Эволюционная - прототип меняется с течением времени, улучшается и преобразовывается

В реальной разработки используют инкрементно-эволюционную модель

3. Водопадная (каскадная) модель.

Состоит из последовательных блоков с возможностью отката. Этапы:

1. Системные Требования
2. требования к ПО
3. Анализ требований
4. Проектирование
5. Разработка
6. Тестирование
7. Эксплуатация

Чем дальше стадия куда надо откатится тем сложнее и ресурснозатратнее будет откат

4. Методология Ройса

Основные нововведения:

1. Do it twice
2. Тестирование
3. Документирование
4. Предварительный дизайн
5. Подключение пользователей

5. Традиционная V-chart model J.Munson, B.Boehm.

В основе лежит по сути водопадная модель, но к каждому уровню добавляется свой уровень тестирования, Модульное, интеграционное и системное. Последний этап - приемочное тестирование. Также тестирование можно разделить на два типа: статитическое и динамическое. Статическое без исполнения программ, а динамическое на выполнение и анализ результата.

6. Многопроходная модель (Incremental model).

За основу берется V-модель со своим уровнем тестирования на каждом этапе. Затем предлагается разбить продукт на несколько отдельных сборок где у каждой сборки

будет свой этап разработки. Интеграция воедино происходит в несколько подходов. Они же называются инкрементами.

7. Модель прототипирования (80-е).

Модель прототипирования — это подход, при котором программу разрабатывают через несколько последовательных прототипов.

Сначала быстро анализируют основные требования и создают упрощённую версию системы. Затем её показывают пользователю и проверяют, насколько она соответствует его ожиданиям. Если что-то не устраивает, создают новый, улучшенный прототип и снова получают обратную связь. Так продолжается, пока требования не станут понятными и пользователя не устроит результат.

8. RAD методология.

Сначала производится проектирование, затем с помощью средств автоматизации собирается ПО. Основным новшеством - является прямое участие пользователя во всех этапах разработки. Бизнес пользователь может сам проектировать и создавать определенные фичи с помощью готовых средств автоматизации.

9. Спиральная модель.

Спиральная модель — это модель разработки ПО, в которой проект развивается **по виткам спирали**, то есть через повторяющиеся итерации.

На каждой ветке определяются риски, на основе этого принимается решение о новой стадии разработки проекта, проводится сверка с картой рисков. Последняя часть витка - это планирование нового витка.

На каждом витке сначала определяют цели и требования, рассматривают возможные варианты реализации и ограничения.

Основное новшество - риски, и подход к новым этапам как к изменениям которые могут быть отклонены или приняты в зависимости от анализа рисков

10. UML Диаграммы: Структурные и поведенческие. Примеры диаграмм.

- Структурные

- Объект

:Мой любимый пирожок

- Класс

Пирожок

Еда
{abstract}

- Интерфейс



Подогреватель

<<interface>>
Подогреватель

- Кооперация (взаимодействие)

Кухня

- Действующее лицо

Бабушка



Бабушка

- Компонент

http HA-proxy

<<component>>
Postgresql

- Артефакт

<<artifact>>
Концепция

<<library>>
makesgood.jar

- Узел

Сервер

Объект - сущность с уникальным набором характеристик

Класс - описание множества объектов с определенными атрибутами и операциями

Интерфейс - набор операций которые предоставляют услуги которые могут быть запрошены потребителем

Взаимодействие - совокупность взаимодействующих между собой объектов

Действующее лицо - внешняя сущность, которая взаимодействует с системой.

Компонент - часть системы с набором интерфейсов

Артефакт - результат выполнения работы системы



Прецедент - акт использования системы действующим лицом,

Состояние - период жизни объекта, который характеризуется различным набором атрибутов

Деятельность - множество действий

Действие - простейшее вычисление

11 хз

12. UML: Диаграмма классов. Элементы диаграммы, пример диаграммы.

Диаграмма классов — это UML-диаграмма, которая показывает **структуру системы**: какие в ней есть классы, какие данные и методы они содержат и как классы связаны между собой.

Класс обычно изображается прямоугольником из трёх частей: имя класса, атрибуты, методы

Основные связи между классами:

- зависимость - один класс временно использует другой.

- ассоциация - один класс связан с другим;
 - композиция - компонент является частью другого и не может существовать без него;
 - агрегация - компонент является частью другого и может существовать без него;
 - включение - компонент является частью другого и имеет доступ к его пространству имен
- обобщение/ наследование - объект является более обобщенной версией другого и может заменять его, также может иметь свои дополнительные особенности
- реализация - исходный элемент реализует контракт какого либо интерфейса
Пример - различные бд, системы заказов и тд

13. UML: Диаграмма состояний. Элементы диаграммы, пример диаграммы.

Блоки - состояния объектов, каждый блок переходит в другой, т е диаграмма отображает изменение состояний объекта через определенные методы и операции. Данная модель достаточно наглядна и позволяет разобраться неподготовленному пользователю

14. UML: Диаграмма размещения. Элементы диаграммы, пример диаграммы.

Показывает как части приложения размещены на серверах и как узлы связаны между собой. Основные элементы: узел, среда выполнения - программная среда внутри узла, артефакт - физический результат разработки, связь - канал между узлами.
нарисовать пример

15. UP методологии (90-е). RUP: основы процесса.

Главная идея — разрабатывать систему **итеративно и поэтапно**, а не пытаться сразу спроектировать и реализовать её целиком

RUP основан на ролях.

Роль - это обязанности которые берет на себя участник рабочего процесса.

Роль чаще всего имеет определенные доступы и ограничения для обеспечения безопасности и разделения обязанностей.

Артефакт - результат деятельности роли. любой продукт труда и тд

Процесс состоит из четырёх фаз: начало, проектирование, построение и внедрение.

16. RUP: Фаза «Начало».

Определяет границы проекта, его ценность, возможные затраты различных ресурсов, а также какие проблемы пользователя может решить проект.

Каждая фаза заканчивается вехой:

это процесс принятия решения о переходе на следующую фазу, либо решение продолжать работать в этой фазе. Решение о переходе на следующую фазу принимается стейкхолдерами(руководством и инвесторами). на данном этапе стороны договариваются о сроках целях приоритетах стоимости и требованиях

17. RUP: Фаза «Проектирование».

Определяет архитектуру системы. Разрабатывается несколько прототипов которые определяют исполняемую архитектуру, которая реализует некоторые функции системы. На этом этапе проводится тестирование и контроль рисков, переоценка ценностей, уточняются сроки и стоимость

Веха:

Сроки и стоимость соблюдены согласно требованиям, контролируются потраченные средства и ресурсы, что важно для планирования следующей фазы

Сложность в том, что стейкхолдеры - люди бизнесовые, далекие от технологий и понимания деталей.

18. RUP: Фаза «Построение».

Происходит основная разработка программного продукта. К её началу архитектура уже должна быть определена и стабилизирована, поэтому серьёзных архитектурных изменений стараются избегать.

Цели:

- Качественно, дешево и быстро сделать готовый продукт.
- Итеративно и инкрементально выполнять разработку, тестирование и интеграцию.
- Создавать промежуточные версии продукта
- Подготовить продукт и пользователей к последующему внедрению.
- Снижать издержки разработки и рационально распределять работу внутри команды.

Веха:

- достаточно ли стабилен продукт для передачи пользователям;
- готовы ли пользователи и заказчик к его внедрению;
- соответствует ли фактическая стоимость проекта запланированной.

19. RUP: Фаза «Внедрение».

Провести альфа/бета тестирование, сравнить работоспособность нескольких версий с подключением пользователя. Маркетинг, привлечение пользователей.

Веха:

Удовлетворены ли пользователи, успешен ли продукт, провести работу над ошибками, сравнить затраты с начальным этапом, выплаты премий и тд

20 Манифест Agile

Люди и взаимодействие важнее процессов и инструментов

Работающий продукт важнее документации

Сотрудничество с заказчиком важнее согласования условий контракта

Готовность к изменениям важнее плана

Основная идея agile это выполнение требований заказчика а также сокращения труда разработчиков на любые сферы, которые не касаются кода

Наиболее известная agile методология это scrum.

21. Scrum.

Одна из известных agile методологий. Обычно используется для маленьких команд. Основным источником требований является бэклог. По сути в бэклоги помещаются задачи с оценкой трудоемкости работы. Работа производится по спринтам, один спринт - 2-4 недели. В конце каждого спринта подводят итоги и демонстрируют работу заказчику. Команда 3-10 человек, также есть владельцы продукта который обычно определяет требования в бэклог и определяет порядок разработки и есть scrum master.

Утром всегда проходят встречи с отчетами и планированием к следующей встрече

22. Disciplined Agile 2.X (2013).

Она основана на RUP и Scrum. Жизненный цикл делится на три фазы: Начало, Построение и Внедрение, а разработка внутри них выполняется короткими итерациями. В отличие от обычного Scrum, DAD охватывает не только разработку, но и вопросы архитектуры, управления, поддержки, эксплуатации и постоянного улучшения процесса.

Основная идея распространить scrum на большую команду состоящей из сотни и тысячи программистов

23. Требования. Иерархия требований. Примеры формулировок на разных уровнях.

Условие которому должна соответствовать система, указывает лишь на то, что нужно реализовать, а не на то, как. Для определения требований используют SRS, модель которая по сути и является тех заданием на разработку. Также в его основу входят варианты взаимодействия пользователя с программой, сценарии работы, включая ошибочные.

Второй способ это use case модель.



Потребности(сделай охуительно) -> функции(чтобы было охуительно сделай то то и то) -> Требования к по(система должна быть uptime 90 процентов времени)

24. Свойства и типы требований (FURPS+).

- Корректность.
- Однозначность.
- Полнота.
- Непротиворечивость.
- Приоритизация.
- Проверяемость.
- Модифицируемость.
- Отслеживаемость.



ИТМО ВТ

FURPS+

- **Functional** — Функциональные требования.
- **Nonfunctional** — Нефункциональные:
 - Usability.
 - Reliability.
 - Performance.
 - Supportability.
- **Дополнительные (+):**
 - Ограничения проектирования / архитектуры.
 - Требования к реализации, интерфейсам.
 - Физические требования.
- **ISO/IEC/IEEE 24765:2010(E) — Systems and software engineering — Vocabulary.**

Grady, R.B., 1992. Practical software metrics for project management and process improvement. Prentice-Hall, Inc.

Функциональные - что система должна делать

usability - то какой экспиренс должна давать пользователю

reability - требования к надежности

performace - требования к производительности полученного по

Supporabilty - требования к условиям поддержки, обновление ПО, выкатка версий и тд
и тп

25. Формулирование требований. Функциональные

Набор функций, возможности ПО, безопасность.

Требованиям задаются уникальные названия FR0. Набор функциональных требований - по сути набор функций, которые должно выполнять ПО

26. Требования к удобству использования и надежности.

Особенности использования ПО юзером, которые нужно учесть при разработке:

- Человеческий фактор, например скорость реакции человека, запрос должен приходить с определенным интервалом, чтобы человек понял что он прошел и не лагать
- UX - user experience, все что касается нативности и очевидности использования логических переходов между частями системы, классификации и расположения объектов
- эстетика

К надежности:

способ, программный алгоритм, потенциал, стандарт



Reliability

- Frequency and severity of failure — частота и обработка отказов.
- Recoverability — способность системы восстанавливать продуктивное функционирование.
- Predictability — предсказуемость поведения.
- Accuracy — точность.
- MTBF — среднее время между отказами.

27. Требования к производительности и поддерживаемости

Производительность:

- скорость решения задачи
- эффективность

- пропускная способность
- время отклика
- время восстановления
- использование ресурсов

Поддерживаемость:

- Расширяемость
- адаптируемость
- совместимость с различным по пользователя
- локализация
- требования к установке, зависимости
- Конфигурируемость, т е ее по сути гибкость

28. Атрибуты требований.



Атрибуты требований

- **Приоритет — MoSCoW:**
 - **MUST have (Minimum Usable SubseT)** — фундаментальные для системы требования.
 - **Should have** — важные.
 - **Could have** — потенциально возможные, улучшающие, к примеру, пользовательское отношение.
 - **Won't have (Would like to have)**— могут быть реализованы в следующих версиях системы.
- Дополнительно используются цифровые приоритеты.

<https://www.stickyminds.com/article/time-boxing-planning-buffered-moscow-rules?page=0%2C0>

Требования способные увеличивать прибыль имеют must have атрибут.

29. Описание прецедента.

Основной сценарий пишут как диалог «актер — система», где каждый шаг имеет наблюдаемый результат. Прецедент описывает действия пользователя в той или иной ситуации, он содержит предусловия, основной поток действий пользователя и альтернативный поток. Т

30. Риски и типы рисков

Риск - событие которое в негативном ключе может повлиять на цели.

Риски прямые не прямые

ресурсные - люди, время деньги

бизнесовые - конкуренция, подрядчики, убыточность решения.

технические - риск выбора определенного по, зависимости и тд

политические - сфера влияния менеджеров

форс мажор

31. Управление рисками: деятельности оценки

Обычно считают ущерб от риска и его вероятность данные значения перемножают и получают оценку данного риска. Обычной шкалы может быть недостаточно

В целом оценка состоит из определения риска, его анализа и приоритизации

32. Управление рисками: контроль и управление

Планирование реакций на риск и мониторинг рисков, если событие наступило риск превращается в проблему. Информироваться стейкхолдеры, корректируется бюджет, а также обеспечивается резерв денег, затем фиксируется опыт ошибок и остаточные риски.

33. Изменение и общая модель управления изменениями

- Контролируемое, журналируемое обновление системы.
- Атрибуты изменения:
 - Идентификатор, дата, ответственный, описание, журнал изменения.
- Управлению изменениями подлежат:
 - Требование новой функциональности.
 - Нефункциональные требования.
 - Исправление дефектов.
 - Другие артефакты архитектуры, анализа, дизайна.

В сложную систему тяжело, а иногда невозможно вносить изменения одновременно, поэтому изменения регистрируют в журнале, где хранится вся инфо о них. Обычно такими журналами являются системы контроля версий.

34. Системы контроля версий. Одновременная модификация файлов

VCS хранит версии, авторов и историю, позволяет создавать ветви и теги, сравнивать и восстанавливать состояния. Современная модель `copy–modify–merge` разрешает нескольким разработчикам менять свои копии параллельно, а затем автоматически объединяет непересекающиеся правки. Если правки затрагивают один участок, возникает конфликт, который решает человек, понимающий намерение обеих сторон. Практики: частые `update/pull`, короткие ветви, маленькие логические `commits`, `review`, CI и тест после `merge`.

35. Subversion: архитектура системы и репозиторий

SVN — централизованная VCS. Сервер хранит единственный репозиторий и глобальные номера ревизий, клиент — рабочую копию с метаданными. Обычно каталоги `trunk`, `branches`, `tags` являются соглашением; дешёвое копирование внутри репозитория создаёт ветку или тег.

Доступ возможен через https, svn, svn+ssh или локальный file; сервером служит svnserve либо Apache с mod_dav_svn. Централизация упрощает единые права и резервное копирование, но commit и получение новых изменений зависят от сервера. Ревизия описывает атомарное изменение всего репозитория.

36. Subversion: основной цикл разработчика и команды

Цикл:

svn checkout URL создаёт рабочую копию

svn status и svn diff используют после правок

svn update получает изменения коллег

svn commit -m "... " публикует атомарную ревизию.

Файлы добавляют и удаляют через svn add, svn delete, перемещают svn move; историю смотрят svn log, автора строки — svn blame, локальную правку отменяют svn revert. svn copy создаёт branch/tag, svn switch переводит рабочую копию, svn merge переносит диапазон изменений. Перед commit рабочая копия должна быть актуальна и проверена.

37. Subversion: конфликты и слияние изменений

Конфликт появляется при update или merge, когда SVN не может совместить правки. Для текста создаются конфликтные варианты и маркеры; для свойств и дерева возможны property/tree conflicts.

Наличие чистого автоматического merge ещё не гарантирует логической корректности.

Разработчик изучает base, свою и входящую версии, формирует правильный итог, собирает и тестирует его, затем выполняет svn resolve --accept working file и commit. Ветку сливают выбранным диапазоном либо reintegrate-стратегией соответствующей версии SVN. Частые небольшие слияния и согласованные границы модулей снижают стоимость конфликтов.

40. Плагин git-flow

Классический git-flow использует main для выпущенных версий и develop для интеграции. feature/ *ответвляются от develop*, release/ стабилизируют выпуск и сливаются в main/develop, hotfix/* начинаются от main и срочно исправляют production. Релизы помечаются tags.

Команды расширения автоматизируют создание и завершение ветвей: git flow init, feature start/finish, release start/finish, hotfix start/finish. Схема удобна при редких версионных релизах и нескольких поддерживаемых версиях. Для частого CD она тяжела: две

постоянные ветви и позднее слияние увеличивают задержку, поэтому часто выбирают упрощённый GitHub Flow или trunk-based.

41. Системы автоматической сборки: предпосылки появления

Каждый раз, чтобы собрать и запустить программу нужно было осуществлять определенный набор команд, решение пришло быстро - скрипты, но любое изменение таким образом нужно было отображать в скрипте что усложняло разработку

Отличия архитектур, скрипт на bash не запустится на винде что логично

Однопроцессорная сборка достаточно долгая

Решение придумали автоматизированные сборки, которые создаются чаще всего автоматически и с минимальным участием разработчика. Также разработаны серверы на которых запускаются такие сборки по типу гитлаба и дженкинс

42. Make и Makefile

Make представляет сборку как граф правил: цель, её зависимости и рецепт. Если цели нет или она старше зависимости, рецепт выполняется, поэтому после изменения файла перекомпилируется только затронутая часть. В Makefile применяют переменные, pattern rules и `$@` (цель), `$<` (первая зависимость), `$^` (все зависимости). `.PHONY` помечает команды вроде `clean`. Make прост и универсален, но рецепты зависят от shell/ОС, а программист обязан точно описать зависимости.

43. Ant и команды Ant

Apache Ant — XML-система сборки Java. В `build.xml` описываются `project`, `properties`, `targets` и `tasks`; `depends` задаёт граф целей. Задачи `javac`, `junit`, `jar`, `copy`, `delete` выполняют типовые операции.

Команды: `ant` запускает `default target`; `ant clean`, `compile`, `test`, `jar` — именованные цели; `ant -Dname=value target` передаёт `property`; `ant -p` выводит цели. Ant гибок, но XML многословен, стандартного ЖЦ и современного встроенного управления зависимостями нет. Содержит переменные которые можно менять в зависимости от проекта

45. Maven: POM, репозитории и зависимости

`pom.xml` содержит координаты, `parent`, `properties`, `packaging`, `modules`, `dependencies`, `build/plugins`, `profiles` и `repositories`. Зависимость задаётся GAV и `scope`; транзитивные

зависимости Maven получает автоматически. Ненужные исключают через exclusions, версии централизуют в dependencyManagement/BOM. Локальный репозиторий — кэш пользователя; удалённые — Central и корпоративный Nexus/Artifactory. Корпоративный проху повышает воспроизводимость и позволяет контролировать лицензии и уязвимости.

46. Maven: структура проекта и GAV

Соглашение Maven: src/main/java, src/main/resources, src/test/java, src/test/resources, pom.xml; результат — в target. Благодаря этому типовой проект почти не требует процедурного скрипта. GAV — groupId:artifactId:version: глобально однозначные координаты артефакта; packaging добавляет тип jar/war/pom. Многомодульный parent POM перечисляет модули и централизует настройки. Координаты позволяют найти бинарный артефакт и его метаданные в любом совместимом репозитории.

47. Maven: зависимости, жизненный цикл и плагины

Основной lifecycle: validate → compile → test → package → verify → install → deploy;
поздняя

фаза включает предыдущие. Фазы исполняются goals плагинов: compiler, surefire/failsafe, jar/war, install, deploy. Scopes compile, provided, runtime, test определяют classpath и транзитивность. Конфликтверсий проверяют mvn dependency:tree. Для CI типична команда mvn clean verify; install кладёт пакет

локально, deploy публикует удалённо.



«Жизненный» цикл

- generate-sources/generate-resources
- compile
- test-compile
- test
- package
- integration-test (pre and post)
- install
- deploy

48. Gradle: преимущества и файл сборки

Gradle строит граф задач; build.gradle использует Groovy, build.gradle.kts — Kotlin. В скрипте объявляют plugins, repositories, dependencies, tasks; Wrapper фиксирует версию Gradle. Сильные стороны: инкрементальность по входам/выходам, build cache, daemon, параллельность и гибкие плагины. По сравнению с Maven меньше жёстких соглашений, но эта свобода требует дисциплины: скрытые side effects делают кэш и сборку невоспроизводимыми.

49. GNU autotools: создание конфигурации

1. Исходный код сканируется, ищутся места в коде которые зависят от особенностей платформы и операционной системы, создается, configure.ac файл
2. Вручную создается makefile.am, где явно указывается как должен быть собран проект и тд и тп
3. autoheader - шаблон config.h, а команда aclocal проверяет зависимости и то что должно быть установлено
4. Затем запускаются automake & autoconf, после которых появляется файл makefile.in и configure.

50 GNU autotools: конфигурация и сборка

Конфигурация содержит в себе:

configure.ac:

Продукт, имя проекта, версии, исходники, заголовки.

Запускается команда `./configure`

Затем проверяется наличие компиляторов и библиотек, запускаются тестовые программы, которые проверяют наличие системных функций и пишут инфо об этом в `config.h`.

Дальше с помощью `make` мы можем собрать проект с помощью `make install`

52. Основные понятия и цели тестирования

- **Mistake (Error)** — ошибка, просчёт (человека).
- **Fault** — дефект, изъян (ПО в результате ошибки).
- **Failure** — неисправность, отказ, сбой (внешнее проявление дефекта).
- **Error** — невозможность выполнить задачу вследствие отказа.
- **BUG** — используется неформально. Может обозначать: дефект, отказ, невозможность выполнить задачу.
 - Что-то другое или ничего не обозначать.

mistake - ошибка разраба

fault - дефект, неверный шаг в алгоритме

failure - сбой, неисправность отказ системы

Error - невозможность выполнить задачу с использованием программы

Bug - неформальное определение для

- Цели тестирования:
 - Обнаружение дефектов.
 - Повышение уверенности в уровне качества.
 - Предоставление информации для принятия решений.
 - Предотвращение дефектов.

Главная цель - увеличение пользовательского доверия в том что программа функционирует корректно при любых обстоятельствах

53. Полное покрытие и его достижимость

Полное покрытие практически не достижимо, само покрытие отражает то насколько в программе предусмотрены все возможные исходы выполнения. полное покрытие чаще всего не нужно

54. Статическое и динамическое тестирование

Статическое тестирование не исполняет программу: review требований/архитектуры/кода, inspection, компилятор, lint, SAST, анализ типов и data flow. Оно рано находит неоднозначности, dead code и потенциальные уязвимости.

Динамическое запускает код и сравнивает результат с oracle: unit, integration, system, acceptance, performance. Подходы дополняют друг друга: статика указывает подозрительный путь, динамический тест подтверждает реальное поведение и временные свойства.

55. Автоматизация и ручное тестирование

Автоматизация тестов - положительное явление, но ручное тестирование иногда дешевле и проще. Сложность авто тестов часто заключается в их написании

Существует регрессионное тестирование - это когда старые тесты запускаются при изменении программы

Чтобы автоматизировать тесты необходимо чтобы у ПО не было вариативности в поведении. Одинаковые данные на вход - одинаковые данные на выход.

56. Источники данных, роли и деятельности

Черный ящик:

подразумевается что внутреннее содержание программы скрыто, работа идет на уровне спецификаций и на основе опыта составителя тестов. Т е система дает данные на вход и сравнивает данные на выходе с ожидаемым поведением

Белый ящик:

Возможно исследовать исходный код. Кроме возможности аналитически проверить корректность кода, данный метод делает возможным оценить размер тестового покрытия. Для этого из приложения строится граф, где части кода представляются как узлы графа, ветвления и циклы — как переходы. С помощью графа можно оценить кол-во обходов и соответственно покрытие программы.

Проектирование тестов. Высокая квала, qa тестировщики

Автоматизация тестов - программисты, специализирующиеся на разработке тестов скриптов или программ. Принято, что модульные тесты пишут сами разработчики исходного кода, а интеграционные и системные тесты создают отдельно выделенные программисты с глубокими представлениями об архитектуре и взаимодействии различных частей приложения.

Непосредственное исполнение тестов - низкая квала

Анализа результатов тестов- высокая квалификация персонала.

57. Тестовый случай и сценарий

Случай - набор входных значений. Предусловий выполнения, ожидаемых результатаов и постусловий. Все это определено в идеале до запуска тестов

Тестовый случай должен быть повторяемым т е одинаковый набор входных значений приводит к одинаковому набору выходных значений.

Тестовый сценарий - последовательность тестовых случаев. В тестовом сценарии как раз таки можно использовать разные тестовые случаи. Кол-во таких сценариев как раз определит тестовое покрытие системы

В тестовых сценариях должны быть отработаны появления и отрицательных и положительных реакций системы на действия пользователей

58. Выбор покрытия и числа тестов. Эквивалентность

Нельзя тестировать вечность, оптимальное число тестов можно определить разными методами:

- Анализ граничных значений

- Таблица решений
- Таблицы переходов
- Сценарии использования

Эквивалентность позволяет определить участки программы где она ведет себя идентично. Чем на меньшее кол-во участков можно разбить, тем меньше тестовых случаев есть в программе. Отдельные тесты для границ составляются

59. Модульное тестирование и JUnit 4

Тестирование отдельных компонентов системы, модуль выполняет некую законченную функцию и его можно тестировать отдельно от всей системы. Модули определены в дизайне программы

Для модульного тестирования юзается:


JUnit

- JUnit фреймворк обеспечивает:
 - Аннотации для маркировки метода как теста `@Test`.
 - Аннотации для маркировки действий до и после теста `@Before`, `@After`, `@BeforeClass`, `@AfterClass`.
 - Методы для проверки (assertion).
 - UI, журнал тестов...

60. Интеграционное тестирование и стратегии

Производится после модульного тестирования и проверяет взаимодействия модулей между собой, api calls, БД, пользовательские тесты, ui

Последовательность интеграции модулей определяется стратегией:

- сверху - вниз: сначала тестируется бизнесовая логика, затем ui, затем подключаются блоки хранения данных, затем тесты модульные и интеграционные

- снизу - вверх: используется когда приложение связано с аппаратурой, интеграция начинается с блоков нижнего уровня
- e2e - по функциям
- backbone - сначала минимальный функционал, затем доп фичи
- big bang - все вместе и молимся

61. Функциональное тестирование и Selenium

Проверяется функционал программы, можно автоматом можно вручную в ui. Внесение изменений в приложение чаще всего также вынуждает менять и тесты. Потому что добавление нового функционала может его сломать. Selenium одно из дополнений к браузеру firefox. Оно позволяет сначала записать тестовую последовательность в виде тестовой программы, а затем использовать ее в другом браузере

62. Статические техники и анализ кода

Проводится перед динамическим. Основная идея в неисполнении кода, а в обнаружении дефектов. Статические техники чаще всего находят причины сбоя а не сами отказы. Существуют различные техники анализа кода:

- Человеческое review
- технический анализ
- Сквозной контроль
- Инспекции

63. Системное и производительности тестирование

Сначала тестируются возможности ПО, стабильность работы системы, затем сценарии усложняются, могут проверяться нетривиальные тестовые сценарии которые возможны при большом наплыве пользователей. Затем проверяется устойчивость к сбоям, вводятся заведомо неверные данные и проверяется корректность реакции системы на такие ошибки

Затем проверяется совместимость на различных системах и кроссплатформенность

Производительность:

- Время ответа
- Точность
- Латенси
- определяется среднее время до отказа и среднее время после восстановления

- Пропускная способность

Чаще всего используют нагрузочное тестирование, где могут например создать большой трафик на приложение, увеличить среднее время ответа и тд

64. Alpha-, beta- и приёмочное тестирование

Альфа тестирование - доступ дают внутренним командам, разработчикам и тестировщикам

Бета-тестирование - Внешние пользователи, добровольцы или ЦА в реальных условиях на пользовательских устройствах.

- **Готовность продукта:** Примерно 90–95%. Продукт стабилен, крупные баги исправлены, инфраструктура готова к нагрузкам.
- **Главная цель:** Собрать отзывы реальных людей, найти мелкие недоработки, оценить удобство (юзабилити) и проверить поведение системы при массовом использовании.

Приёмочное тестирование (Acceptance Testing) — это финальный этап проверки программного обеспечения, который определяет, соответствует ли готовый продукт бизнес-требованиям, ожиданиям заказчика и готов ли он к передаче в эксплуатацию

65. Аспекты быстродействия и влияние измерения



От чего зависит скорость выполнения программ?

- Системный и архитектурный аспект:
 - Архитектура (монолитная, n-tier, потоковая...).
 - Виртуализация и кластеризация.
- Низкоуровневый аппаратный аспект:
 - Частота, количество процессоров, количество ядер.
 - Объем и скорость кэшей, памяти, дисковой подсистемы.
- Программный аспект:
 - Используемые алгоритмы и структуры данных (пузырьковая сортировка vs Qsort).
 - Пулы, многопоточность и блокировки.
- Человеческий фактор (ошибки).

Средства измерения могут быть различны:

- неинтрузивные - не влияют на нагрузку
- слабоинтрузивные
- интрузивные

66. Ключевые характеристики производительности



Основные характеристики и ключевые понятия

- Время отклика, время полного обслуживания и пропускная способность.
- Утилизация (%util или %busy) и (%wait) ожидание ресурса.
- Точка насыщения и масштабируемость.
- Эффективность:
 - CPU на 100% занят, это не значит, что система функционирует эффективно.
- Ускорение и прирост производительности.
- Java: Скорость запуска и memory footprints.



67. Нисходящий поиск узких мест

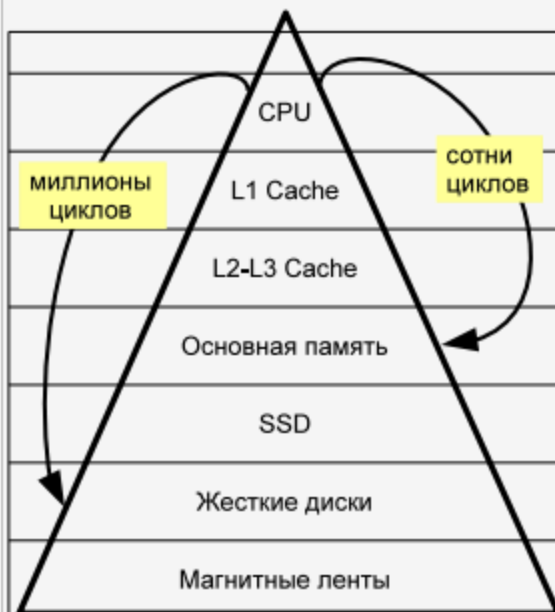
Классический "нисходящий" метод поиска узких мест



- Исключение ошибок аппаратуры и администратора:
 - Системные журналы, файлы конфигурации системы и прикладного ПО.
- Общесистемный и межузловой мониторинг:
 - CPU/память, подсистема IO, дисковые массивы, сеть, каналы передачи, операционная система, виртуализация...
- Мониторинг приложения:
 - Алгоритмические проблемы, проблемы API, многопоточность, блокировки, синхронизация.
- Мониторинг микроархитектуры:
 - Выравнивание данных, оптимизация кэшей, пузырьки конвейера, предсказание переходов.

68. Пирамида памяти

Пирамида памяти



	Объем	Тд	*	Тип	Управл.
CPU	100-1000 б.	<1нс	1с	Регистр	компилятор
L1 Cache	32-128Кб	1-4нс	4с	Ассоц.	аппаратура
L2-L3 Cache	0.5-32Мб	8-20нс	30с	Ассоц.	аппаратура
Основная память	0.5Гб-4Тб	60-200нс	3-10м	Адресная	malloc, free slab-alloc
SSD	128Гб-1Тб/drive	25-250мкс	5д	Блочн.	open read/write
Жесткие диски	0.5Тб-4Тб/drive	5-20мс	4мес	Блочн.	open read/write
Магнитные ленты	1-6Тб/к	1-240с	200л	Последов.	программно

NB: при 2% cache miss производительность падает ниже 60% от идеальной!

Собственно кол-во обращений к памяти на разных уровнях влияет на

производительность системы, чем меньше запросов к памяти тем лучше и быстрее будет работать программа

69. Мониторинг процессов

Мониторинг процессов — это наблюдение за работой ОС и запущенных программ с целью оценки их производительности и поиска проблем.

Основные параметры, которые отслеживаются:

- **CPU** — загрузка процессора в user/system mode, idle, переключения контекста, прерывания, **load average**.
- **Ввод-вывод (I/O)** — объём и количество операций чтения/записи, время ожидания (**wait time**).
- **Память** — свободная память, buffers/cache, операции **page in/page out**.
- **Сеть** — входящий (**RX**) и исходящий (**TX**) трафик, ошибки и коллизии.
- При необходимости собираются более детальные данные отдельных подсистем ОС.

Мониторинг обычно **неинтрузивен**: ОС и так ведёт большинство этих счётчиков, поэтому средства мониторинга в основном просто считывают и анализируют их.

Главная цель: понять, какой ресурс является узким местом — процессор, память, диск или сеть — и определить причину снижения производительности.

70. Мониторинг виртуальной памяти

Виртуальная память — механизм, при котором процесс работает с виртуальным адресным пространством, а ОС отображает его страницы либо в **физическую RAM**, либо при необходимости в **swap** на диске.

При мониторинге виртуальной памяти обычно смотрят:

- **free** — объём свободной оперативной памяти;
- **buffers/cache** — память, используемая ОС под буферы и файловый кэш;
- **page in / page out** — загрузка страниц в RAM и выгрузка из неё;
- **scan rate** — скорость поиска ОС страниц, которые можно освободить;
- **swap in / swap out** — обмен страницами между RAM и swap;
- **page faults** — обращения к странице, которой сейчас нет в нужном отображении.

Page fault бывает:

- **minor** — страницу можно получить без чтения с диска;

- **major** — страницу необходимо загрузить с диска, поэтому он значительно дороже.

Высокий **scan rate**, активный swar и большое количество **major page faults** могут говорить о недостатке оперативной памяти.

В Linux статистику виртуальной памяти можно посмотреть, например, командой **sar -B**.

71. Буферизированный файловый I/O

Буферизированный файловый ввод-вывод — это работа с файлами через промежуточный **буферный кэш ОС**, а не напрямую с диском.

При чтении обычно происходит следующая цепочка:

процесс → **fread()** → **системный вызов read()** → **VFS** → **файловая система** → **buffer cache** → **драйвер** → **диск**

Если нужные данные уже находятся в **buffer cache**, ОС может вернуть их сразу, без обращения к физическому диску. Это значительно ускоряет I/O.

Для файла ОС через **VFS** определяет нужную файловую систему, преобразует имя файла в соответствующий **inode**, после чего получает необходимые блоки данных.

Основные характеристики дискового I/O:

- **r/s, w/s** — количество операций чтения и записи в секунду;
- **rkB/s, wkB/s** — объём прочитанных и записанных данных;
- **avgreq-sz** — средний размер запроса;
- **avgqu-sz** — средняя длина очереди;
- **await** — среднее время ожидания операции;
- **%util** — степень загрузки устройства.

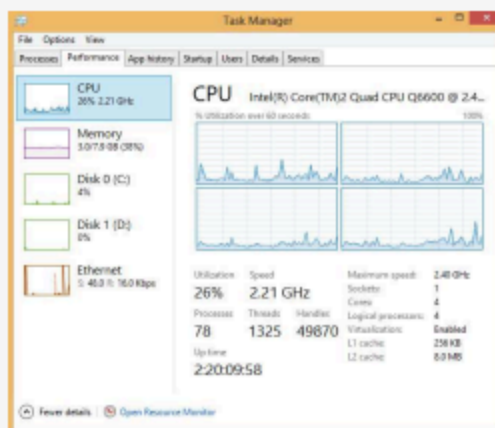
Статистику можно посмотреть командой **iostat**.

Последовательный доступ обычно быстрее, особенно для HDD, поскольку требует меньше перемещений головки диска.

72. Мониторинг Windows и Linux

Мониторинг Windows

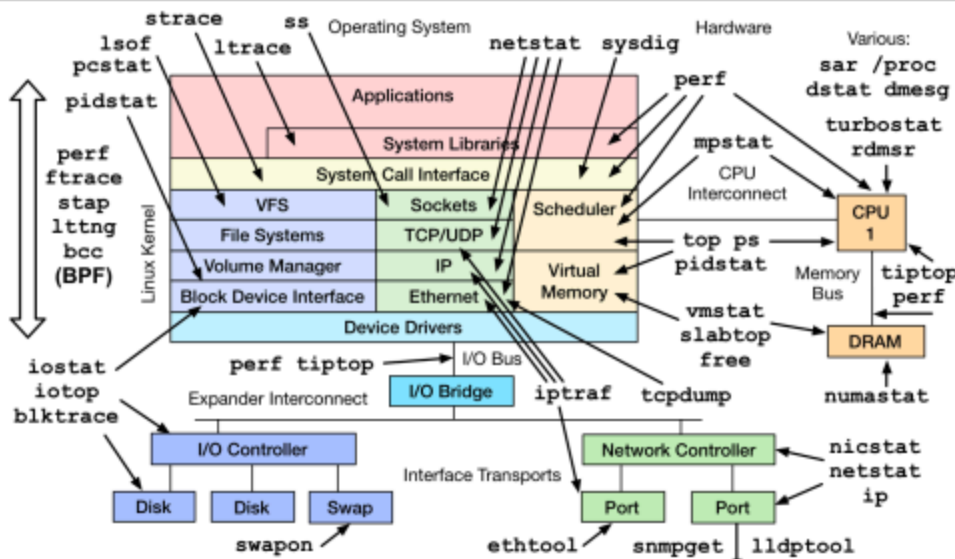
- Встроенные:
 - Task Manager.
 - Resource Monitor & Performance Monitor.
 - Reliability Monitor.
 - **Microsoft SysInternals.**



- Коммерческие и свободно распространяемые:
 - <https://blog.serverdensity.com/windows-monitoring-tools/>

Мониторинг Linux

Linux Performance Observability Tools



<http://www.brendangregg.com/linuxperf.html>

<http://www.brendangregg.com/linuxperf.html> 2016

73. Linux «за 60 секунд»



Системный анализ "за 60 секунд"

- uptime — load average за 1, 5, 15 минут.
- dmesg | tail — последние ошибки.
- vmstat 1 — есть ли свободная память, paging, распределение CPU.
- mpstat -P ALL 1 — распределение по CPU.
- pidstat 1 — статистика по процессам, горячие процессы .
- iostat -xz 1 — параметры ввода-вывода.
- free -m — проверка исчерпания кэшей/буферов.
- sar -n DEV 1 — сетевая статистика по интерфейсам.
- sar -n TCP,ETCP 1 — сетевая статистика по соединениям.
- top — онлайн-мониторинг параметров.



http://www.brendangregg.com/Articles/Netflix_Linux_Perf_Analysis_60s.pdf

74. Тестовая нагрузка и нагрузчики

Рассказать про ДЕВ в нем можно использовать интрузивный мониторинг
Инструменты: JMeter, Gatling, k6, Locust; wrk/ab для простого HTTP; Selenium — лишь для небольшого числа browser flows. Генератор контролируют отдельно: его CPU, сеть или порты не должны стать ложным пределом сервера.

75. Профилирование приложений

Профилирование приложений — это анализ работы программы для поиска узких мест и проблем с производительностью.

Профилировщик позволяет определить:

- сколько времени выполняются функции и методы;
- сколько объектов создаётся в памяти;
- как работают потоки;
- где возникают блокировки и **lock contention**.

Есть два основных подхода:

1. **Инструментирование** — в нужные функции добавляются диагностические точки, которые собирают данные о времени выполнения и других параметрах.

2. **Sampling** — профилировщик периодически прерывает программу и смотрит её состояние, например **стек вызовов**. По количеству попаданий в функции определяется, где программа проводит больше всего времени.

Sampling обычно менее инвазивен и создаёт меньшие накладные расходы.

Главная цель профилирования — найти конкретные функции или участки кода, которые замедляют приложение или неэффективно используют ресурсы.

76. Компромиссы производительности

Компромиссы производительности — это ситуации, когда улучшение одного параметра системы приводит к ухудшению другого. Поэтому нельзя просто «максимально ускорить всё» — приходится выбирать баланс под конкретную задачу.

Основные компромиссы:

- **Скорость ↔ память** — например, кэширование ускоряет работу, но требует дополнительной памяти.
- **Latency ↔ throughput** — можно уменьшать время ответа одного запроса или оптимизировать систему на обработку большого числа запросов в целом.
- **CPU ↔ I/O** — данные можно пересчитывать процессором либо хранить готовый результат и читать его с диска/из сети.
- **Параллелизм ↔ синхронизация** — больше потоков может ускорить работу, но появляются блокировки, переключения контекста и конкуренция за ресурсы.
- **Производительность ↔ надёжность/простота** — агрессивные оптимизации часто усложняют код и могут повышать вероятность ошибок.

Главный принцип: **оптимизировать нужно только после измерений и профилирования**, определив реальное узкое место системы.

То есть задача оптимизации — не получить максимальное значение одного показателя, а найти подходящий **баланс производительности, ресурсов и сложности системы**.

77. Высокий процент SYS

Высокая нагруженность CPU

Причины:

1. Высокая нагрузка на системы ввода/вывода
Решение: реже читать/писать и использовать буфер
2. Недостатки в работе планировщика, избыточное переключение контекстов процессов

Решение: проверить кол-во потоков и что делает в это время ОС

3. Избыточная подкачка страниц и проблемы с виртуальной памятью

Решение: Можно выдать больше памяти за счет процессов, запрет на SWAP и тд

4. Другие системные функции ядра

Решение: Исключить и настроить ядро

78. Высокий процент IO wait

1. Проблемы приложений

оптимизировать операции чтения и письма на диск, стараться объединять несколько операций

2. Буфера/кэши

Расширить память и настроить кэши и буфера

3. Аппаратура

Купить новую

79. Высокий процент Idle

Высокое время простоя CPU

1. Мало процессов на выполнение

Параллелизм, многопоточность

2. Проблемы ОС

80. Высокий %user

Высокий %user означает, что процессор большую часть времени выполняет код пользовательских приложений, а не ядра ОС.

Если CPU загружен почти на 100%, это не обязательно проблема. Важно понять, **эффективно ли приложение использует процессор**. Для этого применяют профилирование.

Основные способы оптимизации:

- использовать алгоритмы с меньшей вычислительной сложностью;
- не создавать одни и те же объекты повторно, а переиспользовать их;
- избегать **busy waiting** и активных опросов в циклах;
- уменьшать **cache miss** и **TLB miss**, правильно организуя данные в памяти;
- учитывать ограниченный размер CPU-кэша и выравнивание данных;
- использовать более лёгкие примитивы синхронизации, где это возможно;

- привязывать потоки к определённым ядрам CPU (**CPU affinity / binding**), чтобы лучше использовать локальный кэш;
- оптимизировать критические участки на уровне компилятора или ассемблера;
- при необходимости использовать больше ядер, GPU или другие ускорители.

Главная идея: при высоком `%user` нужно искать наиболее CPU-затратные участки программы и оптимизировать именно их, а не просто снижать загрузку процессора.